Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

JAVA ASSIGNMENT

NAME:Kavya Gada

SAP ID:60017230102

ROLL NO.:A018

BRANCH:AIML

BATCH:A1

DATE OF SUBMISSION:17/04/24

NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Kavya Gada
A018
A-1

Java Exp - 09

* AIM:- To implement multiple inheritance using interface and method overridding.

* Description of Experiment:-

1) Uses of Interfaces

→ i) It is used to achieve abstraction.
ii) By interface, we can support the functionality of multiple inheritance
iii) It can be used to achieve loose coupling.
iv) Declaring methods that one or more classes are expected to implement.

2) Properties of Interfaces:-
1) An Interface is implicity abstract. You do not need to use abstract keyword while declaring an interface.
2) Each method in an interface is also implicity abstract, so the abstract keyword is not needed.
3) Methods in an interface are implicity public.

3) Overiding and Dynamic polymorphism (dynamic method dispatch)

→ If a sub class has the same name method as declared in the parent class, it is known as Method Overridding. In other words, if a subclass provides the specific implementation of the method that has been declared by one of its parent Class, it is known as method Overridding.

Sundaram
FOR EDUCATIONAL USE

NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

[Type here]

2

Runtime Polymorphism and Dyamic Method Dispatch is a process in which a call to an overidden method is resolved at runtime rather than compile time. In this process, an overridn method is called through the reference variable of a superclass. The determination of method to be called is based on the object being reffered to by the reference variable.

4) Garbage collection (finalize ()).

→ Garbage collection is a process of reclaiming the runtime unused memory automatically. In other words, it is a way to destroy the unused objects. The finalize () method is invoked each time before the object is garbage collected. This method is defined in object class as:-

```
protected void finalize () {
  // Body....
}
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

a) a) Design an interface with a method reversal. This method takes a string as input and return the reversed string. Create a class that implements the above interfaces.

program:-

```
import java.util.*;
interface Reverse {
    String Rev;
}
class StringReverse implements Reverse {
    public String Rev() {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter a string: ");
        String str1 = sc.nextLine();
        StringBuffer sb = new StringBuffer(str1);
        String str2 = sb.reverse().toString();
        return str2;
    }
}

class InterfaceString {
    public static void main(String[] arg) {
        Reverse r = new StringReverse();
        String str = r.rev();
        System.out.println("Reversed String: " + str);
    }
}
```

FOR EDUCATIONAL USE

NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Experiment No: 9

Q1) Design an interface with a method reversal. This method takes a string as input and returns the reversed string. Create a class that implements the above interface.

CODE:

```
interface StringReverser {
	String reverse(String input);
}

class StringReverserImpl implements StringReverser {
	public String reverse(String input) {
	StringBuilder reversed = new StringBuilder(input);
	return reversed.reverse().toString();
	}
}
public class Q9a {
		public static void main(String[] args) {
		String input = "Rishee";
		StringReverser stringReverser = new StringReverserImpl();
		String reversed = stringReverser.reverse(input);
		System.out.println("Reversed string: " + reversed);
	}
}
```

OUTPUT :

NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

```
Microsoft Windows [Version 10.0.19045.4291]
(c) Microsoft Corporation. All rights reserved.

C:\Users\lenovo>cd C:\java program

C:\java program>javac Q9a.java

C:\java program>java Q9a
Reversed string: !olleH

C:\java program>_
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

Q2) WAP to implement three classes namely Student, Test and Result. Student class has member as rollno, and read(). Test class has members as sem1_marks and sem2_marks and read(). Result class has member as total. Create an interface named sports that has a member score (). Derive Test class from Student and Result class has multiple inheritances from Test and Sports. Total is formula based on sem1_marks, sem2_mark and score. Use super keyword.

CODE:

```
interface Sports {
int score();
}
class Student {
int rollno;
public void read() {}
}
class Test extends Student {
int sem1_marks;
int sem2_marks;
public void read() {
super.read();
}
}
class Result extends Test implements Sports {
int total;
public Result(int sem1_marks, int sem2_marks) {
this.sem1_marks = sem1_marks;
this.sem2_marks = sem2_marks;
}
public int score() {
return 10;
}
public void calculateTotal() {
total = sem1_marks + sem2_marks + score();
}
public void displayTotal() {
System.out.println("Total Marks: " + total);
}
}

public class Q9b {
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

```
public static void main(String[] args) {
Result result = new Result(80, 85);
result.calculateTotal();
result.displayTotal();
}
}
```

OUTPUT :

```
Microsoft Windows [Version 10.0.19045.4291]
(c) Microsoft Corporation. All rights reserved.

C:\Users\lenovo>cd C:\java program

C:\java program>javac Q9b.java

C:\java program>java Q9b
Total Marks: 175

C:\java program>
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

CONCLUSION:

- In summary, Java's interfaces facilitate multiple inheritance by allowing classes to implement multiple interfaces.
- Method overriding, a key feature, enables subclasses to customize behavior inherited from interfaces.
- This approach promotes code reuse, flexibility, and maintainability in Java programming

NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018